

OptiStay Datamart Starting and Setup Guide

Prerequisites

Installing and initializing the database requires Docker Desktop to be installed, and port 3306 must be free in order to create the server. Any MySQL client can be used as user interface for working with the database. For this project, Beekeeper Studio was used.

Getting Started

To begin and set up the database a Docker container needs to be created, acting as a local server for the database to reside in. The Container is set up as follows:

```
docker run --name iu-datamart -e MYSQL_ROOT_PASSWORD=mysecretpassword -p 3306:3306
-v iu-datamart-volume:/var/lib/mysql -d mysql:8.0 --sql-
mode="ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FO
R_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION"
```

The next step is to connect to the server at **localhost:3306** as root, using the password from the Docker command. The default database should be left blank on the first run. After this initial setup, follow the execution order described in the next section.

Database Setup, Table Creation, and File Structure

Execution Order:

```
SCHEMA -> ALTER -> INSERT -> TRIGGER -> CONSTRAINT TESTS, JOINS
```

The provided files were created in a specific order during development. To create the **OptiStayDatamart** and the 23 tables, run the **01_schema.sql** file.

Since changes were made after implementation, and **ALTER TABLE** statements were implemented and tested as well, the **02_alter.sql** file has to be executed after the creation, but mandatorily before the **INSERT** statements.

The dummy data insertion is stored in the **03_inserts.sql** file, in which all **INSERT** commands populate the database with pre-generated dummy data. This file has to be executed after the **ALTER TABLE** file. The dummy data was pre-generated with Python scripts in order to fill the database with data for later constraint tests and joins.

To provide automation and ensure specific data integrity, triggers were implemented in the database within the **04_triggers.sql** file. This file must mandatorily run after the **INSERT** statements, since the HostPayouts rows are already filled with dummy data, ensuring that the database is populated with the required number of entries. Otherwise, the trigger could produce duplicate rows in the HostPayouts table or lead to errors and malfunctions.

Constraint tests and complex JOIN queries are shown in the tests.sql and joins.sql files and can be executed after the previously stated files.

Verification

The verification layer is created via the metadata.sql file, showing the total volume of the database in bytes, the total number of tables in the database and the amount of entities per table

Volume

TotalDataLength	TotalIndexLength
376832	671744

Number of tables

NumberOfTables
23

Number of Entities per Table

TableName	NumberOfEntries
Admins	5
Apartments	20
Hosts	6
Users	20
Bookings	20
HostPayouts	26
Payments	20
Reviews	20
CustomerSupport	20
UserChats	20
Addresses	30
Amenities	20
ApartmentNeighbourhoods	10
ApartmentStyles	10
BookingTypes	10
FeeTypes	14

TableName	NumberOfEntries
PaymentMethods	10
Pictures	56
ApartmentAmenities	89
CoHostAssignments	7
FeePayments	121
Recommendations	20
Wishlists	20